



ACCESS CONTROL & INTRUSION DETECTION

CYS335



INTRUSION DETECTION & PREVENTION SYSTEMS

CHAPTER ONE

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY



INTRUSION DETECTION SYSTEMS

UNIT 1

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY

HISTORY OF INTRUSION DETECTION SYSTEMS

- System administrators performed intrusion detection by sitting in front of a console and monitoring user activities.
- Although effective enough at the time, this early form of intrusion detection was **ad hoc** and not scalable.
- The next step in intrusion detection involved **audit logs**, which system administrators reviewed for evidence of unusual or malicious behavior.
- As storage became cheaper, audit logs moved **online** and researchers developed programs to analyze the data.
- In the early '90s, researchers developed **real-time Intrusion Detection Systems** that reviewed audit data as it was produced.

INTRUSION DETECTION OVERVIEW

- A device that generates alerts when it observes potentially malicious traffic is called an **Intrusion Detection System (IDS)**.
- IDSs can be **software applications** that are installed on endpoints or dedicated **hardware** devices that are connected to the network.
- The goal of intrusion detection is seemingly simple, to **detect intrusions**.
- If there is no **manifestation** or if the information it contains is untrustworthy, then the system cannot detect the intrusion.

DETECTION TECHNIQUES

1. Anomaly Detection

- Creates a traffic **profile** as it observes traffic in normal operation.
- It then looks for packet streams that are statistically unusual.
- The great thing about anomaly-based IDS systems is that they **don't rely on previous knowledge** about existing attacks.
- Anomaly detection uses models of the intended behavior of users and applications, interpreting deviations from this "**normal**" behavior as a problem.
- A basic assumption of anomaly detection is that **attacks differ from normal behavior**.
- The main advantage of anomaly detection systems is that they **can detect previously unknown attacks**.
- In actual systems, however, the advantage of detecting previously unknown attacks is paid for in terms of **high false positive rates**.

DETECTION TECHNIQUES (CONTINUES...)

2. Misuse Detection

- Essentially define what's wrong.
- They contain attack descriptions (or "**signatures**") and match them against the audit data stream, looking for evidence of known attacks.
- The **main advantage** of misuse detection systems is that they focus analysis on the audit data and typically **produce few false positives**.
- The **main disadvantage** of misuse detection systems is that they **can detect only known attacks** for which they have a defined signature.
- As new attacks are discovered, developers must model and add them to the signature database.

DETECTION TECHNIQUES (CONTINUES...)

3. Signature-based Detection

- Maintains a database of attack signatures against which it compares network packets.
- If a packet triggers a match to one of the signatures, the IDS flags it.
- In other words, a signature-based IDS is completely **blind to new attacks** that have yet to be recorded.

DETECTION TECHNIQUES (CONTINUES...)

4. Hybrid Detection Method

- Uses both **Signature** and **Anomaly**-based intrusion detection methods.
- However, the main reason behind the development of a hybrid detection system is to identify more potential attacks with **fewer errors**.

TOP INTRUSION DETECTION SYSTEM TOOLS

1. McAfee

- Is an IDS designed to identify **real-time** malicious activities.
- It uses both **signature** and **anomaly** methods to identify the threats with emulation techniques.
- Therefore, McAfee is a **scalable** application.

TOP INTRUSION DETECTION SYSTEM TOOLS (CONTINUES...)

2. Suricata

- It is an **open-source** tool based on a **Network Intrusion Detection System (NIDS)**.
- It detects identified threats and malicious activity in **real-time**.
- It uses a **Signature-based** method to identify the known threats or intrusion.

TOP INTRUSION DETECTION SYSTEM TOOLS (CONTINUES...)

3. Cisco Stealthwatch

- Cisco designed Stealthwatch with **NIDS** and **HIDS**.
- It is compatible with **Windows**, **Linux**, and **MacOS** operating systems.
- it uses **machine learning** to create baselines of patterns which is acceptable.
- One of the best features of this tool is that it can also detect intrusion and suspicious activities in the encrypted network without decrypting it.

RESPONSE: AFTER THE INTRUSION

- An intrusion detection system's **response** is its output or action upon detecting a problem.
- A response can take many different forms; the most common is to **generate an alert** that describes the detected intrusion.
- There are also more **aggressive responses**, such as **sounding a siren**, or even **mounting a counter-attack**.
- A **counterattack** might include reconfiguring a router to block the attacker's address or even attacking the culprit.
- Obviously, **aggressive responses can be dangerous**, since they could be launched against innocent victims.

TYPE OF THREATS

1. Internal Threats

- The term “**internal attack**” is used to describe an attack being implemented from a person or organization with some level of authorized access on your network.
- Internal attacks are performed from within the trusted area of the network.

TYPE OF THREATS (CONTINUES...)

2. External Threats

- Includes anyone that doesn't have authorized access to the internal network but attempts to gain access.
- External threats are what companies spend the most time and money trying to prevent.

NEED OF INTRUSION DETECTION SYSTEMS (IDS)

1. **Monitor** inbound and outbound network traffic.
2. **Analyze** the patterns in the network continuously.
3. Send an **alarm** immediately after detecting unwanted intrusion and activities in the network.

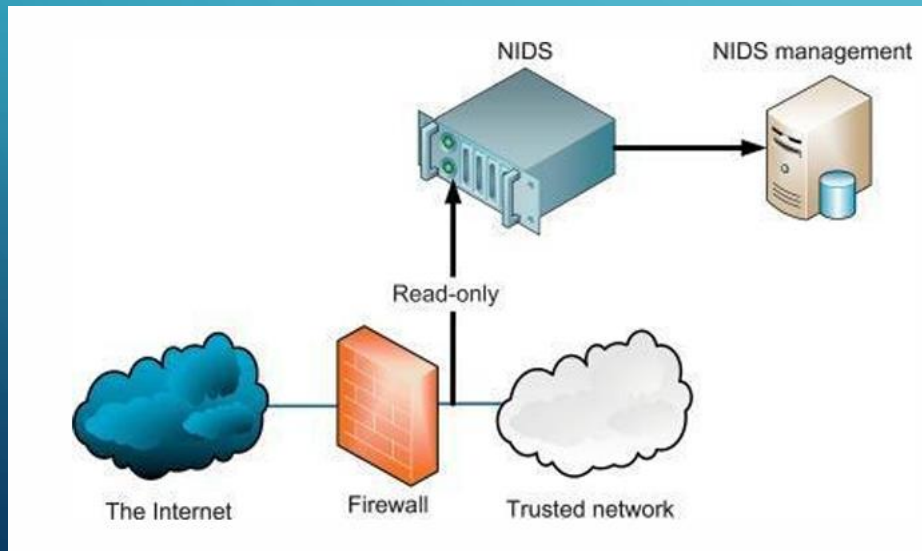
TYPES OF INTRUSION DETECTION SYSTEMS

1. Network Intrusion Detection System (NIDS)

- NIDS **monitors** the traffic on the network from all devices.
- Similarly, it examines the traffic passing on the entire subnet and verifies it with the packet metadata and content.
- If NIDS detects any intrusion in the network, a warning alert is sent to the admin of that network.
- Instead of analyzing information that originates and resides on a computer, network-based IDS uses techniques like “**packet-sniffing**” to pull data from TCP/IP or other protocol packets traveling along the network.

TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

- NIDS usually **require promiscuous network** access in order to analyze all traffic, including all uncast traffic.
- The NIDS sniffs the internal interface of the firewall in **read-only** mode and sends alerts to a NIDS Management server via a different (i.e., read/write) network interface.



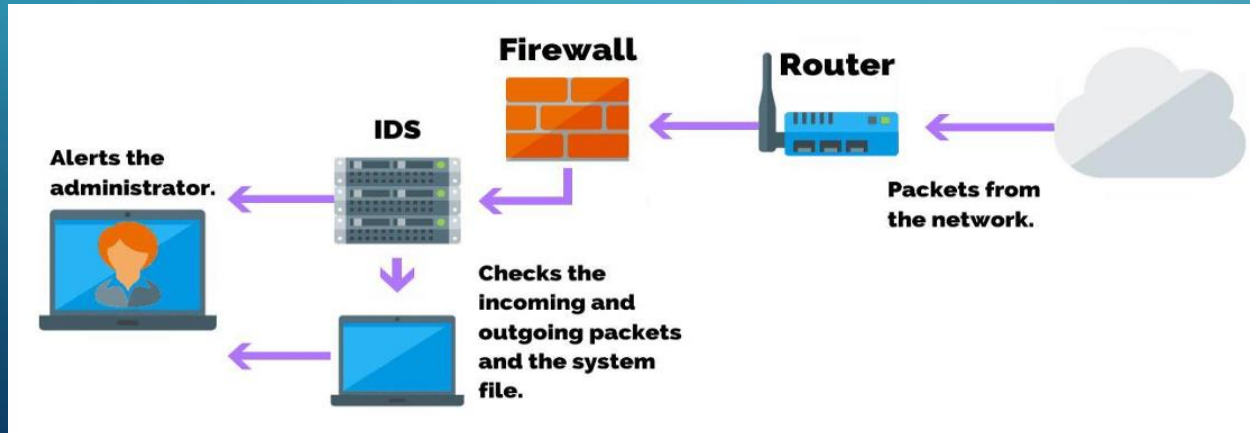
TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

2. Host Intrusion Detection System (HIDS)

- Organizations install a HIDS on independent networked devices.
- HIDS examines the incoming and outgoing traffic of the **device only**.
- It detects suspicious activities on the device and alerts the administrator.
- its network card operates in **no promiscuous mode**.
- No promiscuous mode of operation can be an advantage in some cases, because not all NICs are capable of promiscuous mode.
- Promiscuous mode can be CPU intensive for a **slow** host machine.

TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

- Many host-based IDSs **depend on signature** or string matching to detect threats, and they can be defeated by simply changing a tool enough that the signature no longer matches.
- Host-based systems were the first type of IDS to be developed and implemented.
- These programs are highly **effective for detecting insider abuses**.



TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

3. Protocol-based Intrusion Detection System (PIDS)

- Organizations set up a PIDS at the front end of the server.
- It interprets the protocols between the server and the user.
- PIDS monitors the HTTPS server regularly to secure the web.

TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

4. Application Protocol-based IDS (APIDS)

- Similarly, APIDS is set up within a group of servers.
- It interprets communication with the applications within the server to detect the intrusion.

TYPES OF INTRUSION DETECTION SYSTEMS (CONTINUES...)

5. Hybrid Intrusion Detection System

- is a **mixture** of two different IDS (**host** agents with **network** information).
- Hybrid System is **more responsive** and **effective** as compared to other IDS.



INTRUSION PREVENTION SYSTEMS (IPS)

UNIT 2

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY

24

INTRUSION PREVENTION SYSTEMS

- IPS is a form of network security that works to **detect and prevent** identified threats.
- The IPS reports these events to system administrators and takes preventative action, such as closing access points and configuring firewalls to prevent future attacks.

HOW DO INTRUSION PREVENTION SYSTEMS WORK?

- Intrusion prevention systems work by scanning all network traffic.
- The IPS performs **real-time** packet inspection, deeply inspecting every packet that travels across the network.
- If any malicious or suspicious packets are detected, the IPS will carry out one of the following actions:
 - i. Terminate the TCP session
 - ii. Reconfigure the firewall
 - iii. Remove or replace any malicious content

HOW DO INTRUSION PREVENTION SYSTEMS DETECT MALICIOUS ACTIVITY?

1. Signature-based Detection

- Involves a dictionary of uniquely identifiable signatures located in the code of each exploit.
 - Exploit-facing** – detect malicious activity based on common attack patterns.
 - Vulnerability-facing** – attempt to detect malicious activity by identifying specific vulnerabilities.

HOW DO INTRUSION PREVENTION SYSTEMS DETECT MALICIOUS ACTIVITY? (CONTINUES...)

2. Statistical Anomaly-based Detection

- Detection randomly sample network traffic and then compare the samples to a predetermined baseline performance level.

TYPES OF PREVENTION

1. **Signature-Based** – uses predefined signatures of well-known network threats.
2. **Anomaly-Based** – monitors for any abnormal or unexpected behavior on the network.
3. **Policy-Based** – This approach requires administrators to configure security policies according to organizational security policies and the network infrastructure.

TYPES OF INTRUSION PREVENTION SYSTEM (IDS)

1. **Network-based Intrusion Prevention System (NIPS)** – This type of intrusion prevention system has the ability to **monitor the whole network** and look for suspicious traffic by reviewing protocol activity.
2. **Wireless Intrusion Prevention Systems (WIPS)** – Only monitor wireless networks for suspicious activity by reviewing wireless networking protocols.
3. **Network Behaviour Analysis (NBA)** – Looks at network traffic in an effort to locate threats that cause unusual traffic flows, including distributed denial of service (DDoS) attacks and policy violations.
4. **Host-based Intrusion Prevention System (HIPS)** – Is an installed software package that looks into suspicious activity that occurs within a single host.

ANATOMY OF INTRUSION ANALYSIS

- 1. Pre-Processing** – The data is collected about the intrusions, vulnerabilities and attacks and then it is putted down into classification scheme or pattern descriptors.
- 2. Analysis** – Once the processing is completed, the analysis stage begins. The analysis engine looks for defined patterns that are known as attacks.
- 3. Response** – The inline sensor can provide real time prevention through an automated response.
- 4. Refinement** – This gives the opportunity to reduce false-positive levels and to have a more accurate security tool.

WHAT IS A VULNERABILITY ASSESSMENT?

- A **vulnerability assessment** is the process of defining, identifying, classifying and prioritizing vulnerabilities in computer systems, applications and network infrastructures.
- The process is sometimes referred to as vulnerability assessment/**penetration testing**, or **VAPT**.
- Importance of vulnerability assessments include –
 - i. Provides an organization with details on any **security weaknesses (Vulnerability Identification)**
 - ii. **Assess the risks** associated with those weaknesses (**Risk Assessment**).
 - iii. **Strengthening the Security Posture**

TYPES OF VULNERABILITY ASSESSMENTS

1. **Network-based scans** – are used to identify possible network security attacks.
2. **Host-based scans** – are used to locate and identify vulnerabilities in servers, workstations or other network hosts.
3. **Application scans** – test websites to detect known software vulnerabilities and incorrect configurations in network or web applications.

CREDENTIAL ANALYSIS NON CREDENTIAL ANALYSIS

1. **Credential-based Vulnerability Assessments** – which make use of the admin account, do a more thorough check by looking for problems that cannot be seen from the network.
2. **Non-credentialed Scans** – provide a quick view of vulnerabilities by only looking at network services exposed by the host.



SNORT

UNIT 3

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY

35

INTRODUCTION TO SNORT

- **Snort** is an open-source network based Intrusion Detection System (IDS) which is written in C programming language.
- It was developed in 1998 by Martin Roesch.
- Now it is developed by Cisco.

```
(root@DESKTOP-51DFP8C) - [ /mnt/c/Users/James Soko ]
# snort -V

_*> Snort++ <*-
o" )~
' ' ' '
Version 3.1.82.0
By Martin Roesch & The Snort Team
http://snort.org/contact#team
Copyright (C) 2014-2024 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using DAQ version 3.0.12
Using LuaJIT version 2.1.1700206165
Using OpenSSL 3.5.0 8 Apr 2025
Using libpcap version 1.10.5 (with TPACKET_V3)
Using PCRE version 8.39 2016-06-14
Using ZLIB version 1.3.1
Using LZMA version 5.8.1
```


SNORT FEATURES

- It is based on library packet capture tool.
- Its features include:
 - 1. Real-time traffic monitor** – used to monitor the traffic that goes in and out of a network.
 - 2. Content matching** – collates rules by the protocol.
 - 3. Creates logs**
 - 4. Default and Local Rules**

SETTING UP YOUR HOME NETWORK

- Identify your active network interface.

```
(root@DESKTOP-51DFP8C) - [ /etc/snort ]
# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:17:5f:5b brd ff:ff:ff:ff:ff:ff
    inet 172.24.20.179/20 brd 172.24.31.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe17:5f5b/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

- Alternatively, you can use **ifconfig** and **ip addr**.

SETTING UP YOUR HOME NETWORK (CONTINUES...)

- ❑ Edit the **snort.lua** or **snoft.conf** file using **nano** command.

From

```
-- 1. configure defaults
--
-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = 'any'
```

To

```
-- 1. configure defaults
--
-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = '172.24.20.0/20'
```

PACKET SNIFFING MODE CONFIGURATIONS

- Edit the snort configuration file with **nano snort.lua** or **nano snort.conf**

From

```
ips =
{
    -- use this to enable decoder and inspector alerts
    -- enable_builtin_rules = true,

    -- use include for rules files; be sure to set your path
    -- note that rules files can include other rules files
    -- (see also related path vars at the top of snort_defaults.lua)

    variables = default_variables,
}
```

To

```
ips =
{
    -- use this to enable decoder and inspector alerts
    enable_builtin_rules = true,

    -- use include for rules files; be sure to set your path
    -- note that rules files can include other rules files
    -- (see also related path vars at the top of snort_defaults.lua)

    variables = default_variables,
}
```

SNORT COMMAND LINE OPTIONS

- Run Snort in **packet sniffing mode** using the configuration file.

```
(root@DESKTOP-51DFP8C) - [/mnt/c/Users/James Soko]
# cd /etc/snort/

(root@DESKTOP-51DFP8C) - [/etc/snort]
# snort -c snort.lua -i eth0
```

- c** allows you to specify which configuration file you want to use. In snort v3 **snort.lua** is used while in v2 **snort.conf** is used.
- i** specifies which interface Snort should listen on e.g., Ethernet (**eth0**)

SNORT COMMAND LINE OPTIONS (CONTINUES...)

iii. **-V** displays the Snort **version** number and then exits.

iv. **-?** lists all switches and **options** and then exits.

```
(root@DESKTOP-51DFP8C)-[/etc/snort/rules]
# snort -?
-? <option prefix> output matching command line option quick help (same as --help-options) (optional)
-A <mode> set alert mode: none, cmg, or alert_*
-B <mask> obfuscated IP addresses in alerts and packet dumps using CIDR mask
-C print out payloads with character data only (no hex)
-c <conf> use this configuration
-D run Snort in background (daemon) mode
-d dump the Application Layer
```

SNORT COMMAND LINE OPTIONS (CONTINUES...)

- Snort itself uses something called **Data Acquisition library (DAQ)** to make abstract calls to packet capture libraries.

❑ Run Snort in inline mode (ensure interface and bridge setup if required)

```
(root@DESKTOP-51DFP8C)-[/etc/snort]  
# snort --daq afpacket -c snort.lua -i eth0:lo -Q
```

- **--daq afpacket** tells Snort to use the **AFPacket DAQ module**.
- **-Q** enables **inline mode** (for IPS).

SNORT ALERT MODES

- You can view these snort alert modes via nano `/etc/snort/snort.lua` in v3 and `/etc/snort/snort.conf` in v2.

```
-----  
-- 7. configure outputs  
-----  
  
-- event logging  
-- you can enable with defaults from the command line with -A <alert_type>  
-- uncomment below to set non-default configs  
--alert_csv = { }  
    alert_fast = { }  
--alert_full = { }  
--alert_sfsocket = { }  
--alert_syslog = { }  
--unified2 = { }
```

SNORT ALERT MODES & SNORT COMMAND LINE OPTIONS (CONTINUES...)

1. Fast

```
(root@DESKTOP-51DFP8C)-[/etc/snort]  
# snort -c snort.lua -i eth0 -A fast
```

vi. -A generates an alert using one of the specified alert-modes: **fast**, **full**, **none**, and **unsock**.

- Snort alerts report the **timestamp**, send an **alert message**, show the source and destination **IP address** and **port**.

SNORT ALERT MODES (CONTINUES...)

2. Full

- Is the **default alert mode**.
- Additionally to the information printed in the fast mode, the full mode shows the **TTL**, packet headers and datagram **length**, **service**, ICMP type, window size, **ACK** and sequence number.

3. Console – prints fast alerts in the console.

SNORT ALERT MODES (CONTINUES...)

- 4. None** – With this mode, Snort **does not generate alerts**.
- 5. Unsock** – This is useful to **export alert** reports to other programs through **Unix sockets**.
- 6. Cmg** – This alerts mode was developed by Snort for **testing purposes**; it prints a full alert on the console without saving logs.



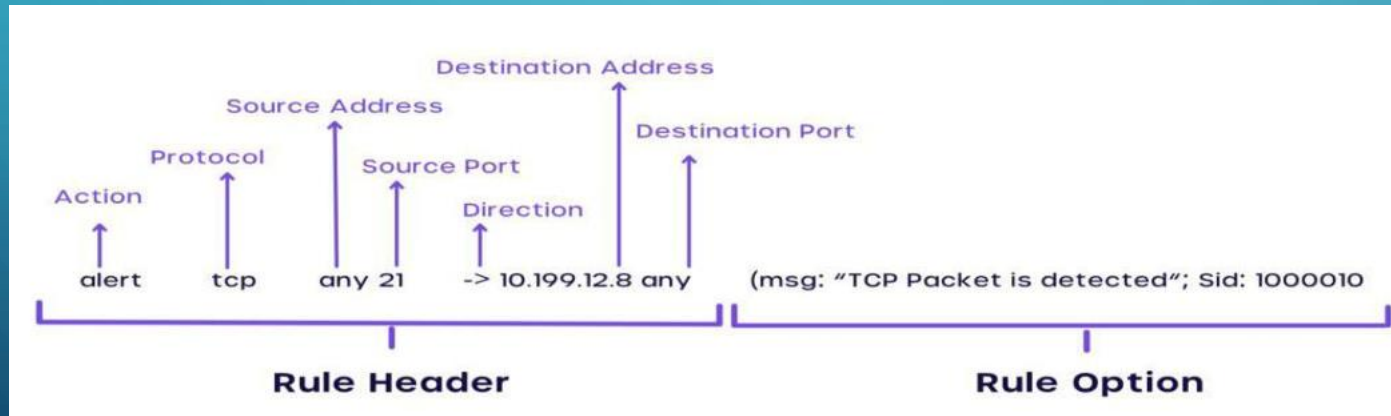
SNORT RULES

UNIT 4

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY

WORKING WITH SNORT RULES

- ❑ Provide a sample Snort rule to detect HTTP traffic on port 80 and explain its structure.
- Navigate to `nano /etc/snort/rules/local.rules`



RULE HEADER

1. Action

- i. **Alert** – alerts the network administrator upon detecting a potential threat.
- ii. Drop
- iii. Pass
- iv. Log
- v. Activate
- vi. Dynamic
- vii. Reject
- viii. Sdrop

RULE HEADER (CONTINUES...)

- 2. Protocol** – you may key in the Protocol address that you need e.g., **tcp**.
- 3. Source IP Address** – For example, **172.24.20.0** or **any**
- 4. Source Port** – computers generally have 65,536 TCP Ports (e.g., **80** for HTTP) and if you wish, you may again key-in '**any**' to define all such ports in the rule.
- 5. Flow** – refers to the **direction** of the traffic and denoted by **->**
- 6. Destination IP Address**
- 7. Destination Port**

WORKING WITH SNORT RULES (CONTINUES...)

- Below is our local rule:

```
GNU nano 8.4                                local.rules *
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert tcp any any -> any 80 (msg:"HTTP Traffic Detected"; sid:1000001; rev:1;)
```

- msg** is the message displayed after the detection.
- sid** is the snort ID (below 1 million for default rules and above that are local rules)
- rev** is the revision number.

SNORT RULES EXAMPLES AND CHEAT SHEET

❑ NOTE:

1. You can replace **any** with an IP Address e.g., **172.24.20.0/24** or port e.g., **21**
2. You can also replace the IP Address **172.24.20.0/24** with **\$HOME_NET**
3. You can also replace any with **EXTERNAL_NET** if specified in the configuration file.

CONFIGURING LOCAL RULES

- Edit the snort configuration file and include the local rule path.

```
ips =  
{  
    -- use this to enable decoder and inspector alerts  
    enable_builtin_rules = true,  
  
    -- use include for rules files; be sure to set your path  
    -- note that rules files can include other rules files  
    -- (see also related path vars at the top of snort_defaults.lua)  
  
    variables = default_variables,  
  
    include = {  
        RULE_PATH .. '/local.rules'  
    }  
}
```

SNORT COMMAND LINE OPTIONS (CONTINUES...)

- ❑ Test your snort configurations for any errors.

```
(root@DESKTOP-51DFP8C)-[/etc/snort]  
# snort -c snort.lua -T
```

```
Snort successfully validated the configuration (with 0 warnings).  
o")~ Snort exiting
```

- vii.-T** starts Snort in self-test mode. Useful for **debugging** Snort before it is run in daemon mode or before it is launched on a production box. Can be used for **testing the correctness** of your configuration files.

THE SNORT CONFIGURATION FILE

- A sample configuration file **snort.conf** is included in the Snort distribution.
- You use the **-c** command line switch to specify the name of the configuration file.
- This file contains six basic sections:
 - 1. Variable definitions** – where you define different variables.
 - 2. Configure parameters** – specify different Snort configuration options.

THE SNORT CONFIGURATION FILE (CONTINUES...)

- 3. Preprocessor configuration** – are used to perform certain actions before a packet is operated by the main Snort detection engine.
- 4. Output module configuration** – control how Snort data will be logged.
- 5. Defining new action types**
- 6. Rules configuration and include files** – These files are then included inside the main configuration file using the **include** keyword.



ANALYSIS CONSOLE FOR INTRUSION DATABASES (ACID)

UNIT 5

NOTICE

- The slides will be added before *Monday*, for now study the lecture slides.



ACCESS CONTROL

CHAPTER TWO

PREPARED BY JAMES SOKO FOR MR. WALUBITA N. © 2025 KAPASA MAKASA UNIVERSITY

60

WHAT IS ACCESS CONTROL?

- Granting or denying approval to use specific resources.

1. Identification – Presenting credentials.

2. Authentication – Checking the credentials.

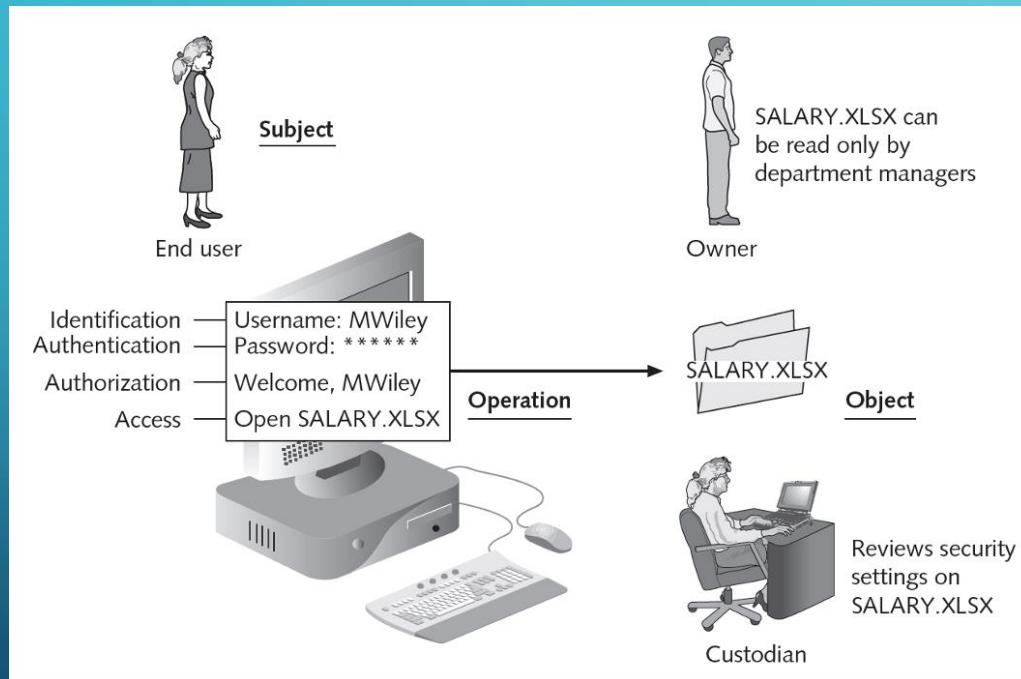
3. Authorization – Granting permission to take action.

4. Access – Right given to access specific resources.

ACCESS CONTROL TERMINOLOGY

- 5. Object** – Specific resource (e.g., file)
- 6. Subject** – User or process functioning on behalf of a user (i.e., computer user)
- 7. Operation** – Action taken by the subject over an object (e.g., deleting a file)

ACCESS CONTROL TERMINOLOGY (CONTINUES...)



8. Owner – Determines the level of security needed to the data.

9. Custodian – Assigned by the owner to review security settings.

10. End User – Accesses the information and does not circumvent security based on the organizations guidelines.

ACCESS CONTROL MODELS

1. Mandatory Access Control (MAC)

- Most restrictive access control model.
- Typically found in military settings.
- It has two elements –
 - i. **Labels** – indicate level of privilege
 - ii. **Levels**
 - MAC grants permissions by matching object labels with subject labels.
 - To determine if file may be opened:
 - i. Compare object and subject labels
 - ii. Subject must have equal or greater level than object to be granted access

ACCESS CONTROL MODELS (CONTINUES...)

- Two major implementations of MAC
 - Lattice model** –
 - Bell-LaPadula** – Subjects may not create a new object or perform specific functions on lower level objects.

ACCESS CONTROL MODELS (CONTINUES...)

2. Discretionary Access Control (DAC)

- Least restrictive model.
- Every object has an owner.
- Owners have total control over their objects.
- Owners can give permissions to other subjects over their objects.

ACCESS CONTROL MODELS (CONTINUES...)

3. Role Based Access Control (RBAC)

- Also called **Non-discretionary Access Control**.
- Access permissions are based on user's job function.

4. Rule Based Access Control (RBAC)

- Dynamically assigns roles to subjects based on a set of rules defined by a custodian.

BEST PRACTICES FOR ACCESS CONTROL

- 1. Separation of duties** – Fraud can result from single user being trusted with complete control of a process.
- 2. Job rotation** – Limits amount of time individuals are in a position to manipulate security configurations.
- 3. Least privilege** – Helps reduce attack surface by eliminating unnecessary privileges.
- 4. Implicit deny** – If a condition is not explicitly met, access request is rejected.

ACCESS CONTROL LISTS

- Set of permissions attached to an object.
- When subject requests to perform an operation, system checks ACL for an approved entry.
- Each entry in the ACL table is called Access Control Entry (ACE).

NOTICE

- These slides are not complete but almost done, for now refer to the lecture slide.